



DEPARTMENT OF COMPUTING
AND CONTROL ENGINEERING

ДЕПАРТМАН ЗА РАЧУНАРСТВО
И АУТОМАТИКУ

ACSE

Odsek za automatiku, geomatiku
i upravljanje sistemima

Sub- department for Automatic Control
and System Engineering

Veštačke neuronske mreže (VNM)

Artificial Neural Networks (ANN)

Modeliranje i simulacija sistema

Mesto VNM

ML/VNM koristi podatke u algoritamski definisanom procesu dobijanja modela izabranog tipa.

- dizajniraju se algoritmi i statistički modeli na osnovu podataka, a bez eksplicitnog programiranja.
- rešavaju se brojni problemi (naučni i inženjerski) gde treba pronaći informaciju na osnovu složenih i nesigurnih podataka.

Identifikacija i ML se preklapaju i upotpunjuju

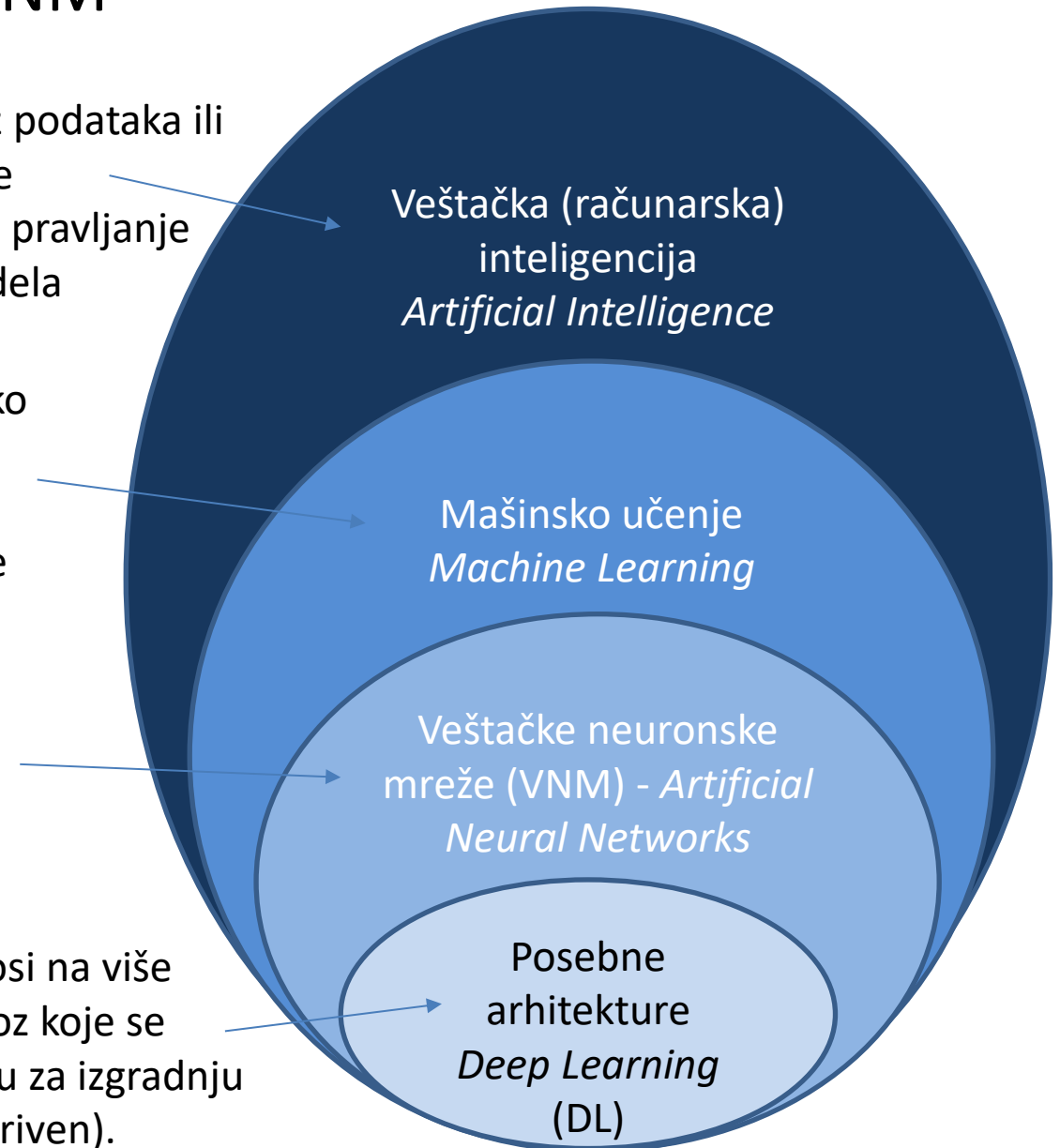
- identifikacija pravi precizne matematičke modele za predviđanje ponašanja sistema i upravljanje

Načini učenja iz podataka ili iskustva da bi se automatizovalo pravljanje analitičkog modela

Uključuje ljudsko ponašanje i inteligenciju u mašine/sisteme

Izračunavanje kroz višeslojnu neuronsku mrežu

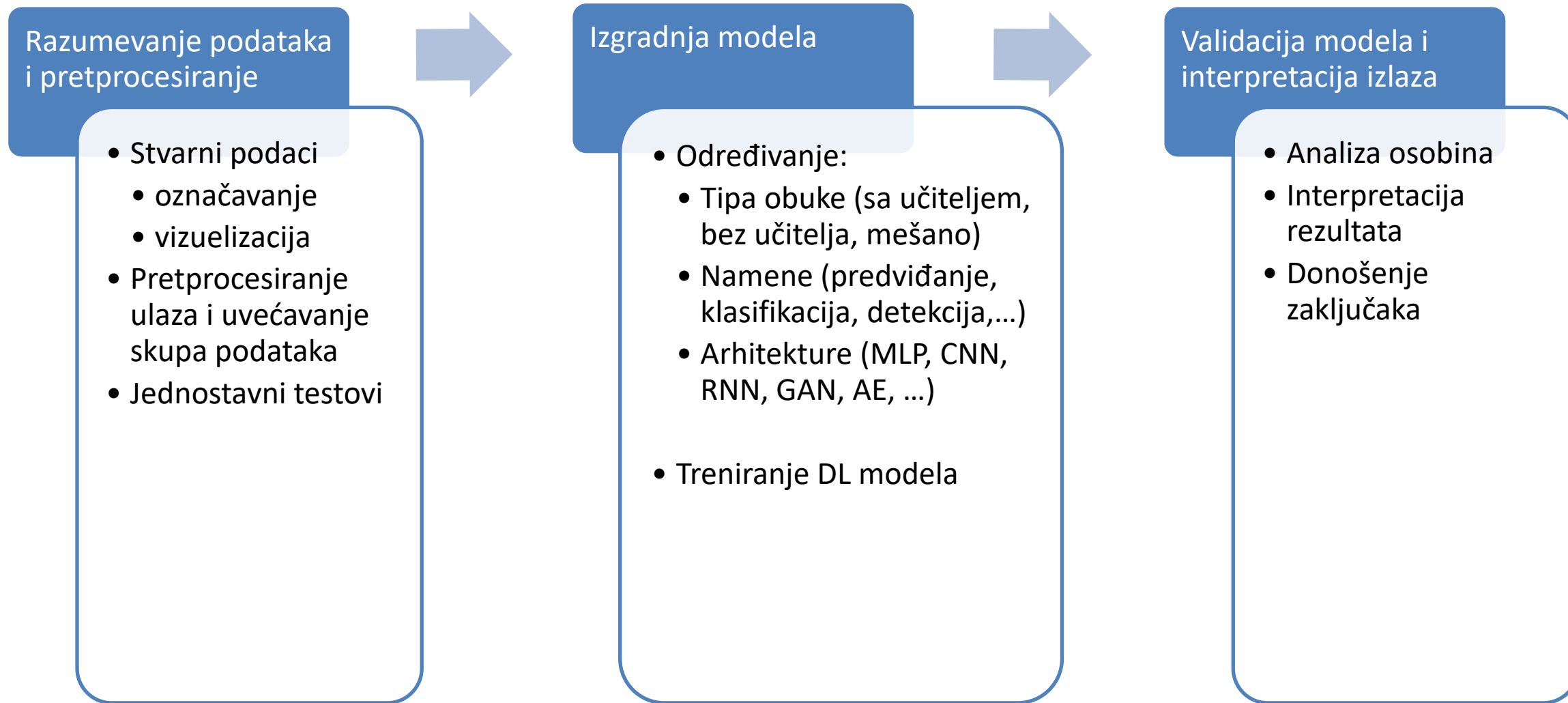
“Deep” se odnosi na više nivoa ili faza kroz koje se podaci obrađuju za izgradnju modela (data-driven).



Istorijat

- Kasne 1980-te: brojne arhitekture VNM i efikasno učenje
 - višeslojni perceptron i backpropagation, samoorganizujuće mape, radijalne bazne funkcije, ...
- 2006. DL - efikasan za velike količine podataka
 - značajan uspeh u klasifikaciji i regresiji
 - iako je obuka modela spora, upotreba obučenih modela je brža u odnosu na druge ML tehnika
- 2012. DL primena u računarskoj viziji (*Computer Vision*)
 - omogućena jakim hardverom (# tranzistora u CPU $\sim 10^9$, primena GPU) i veliki obučavajući skupovi podaka (npr. IMAGENET $\sim 10^{14}$ pixels)
- Skorije
 - obrada prirodnih jezika i transformacije (*Large Language Models*): GPT, BERT, ...
 - brojna unapređenja i primene:
 - u zdravstvu, analizi raspoloženja, vizuelnom prepoznavanju, poslovnoj inteligenciji, „pametnim“ gradovima, računarskoj bezbednosti, ...
 - 4.0 i 5.0 industrijske revolucije, uglavnom zasnovana na DL

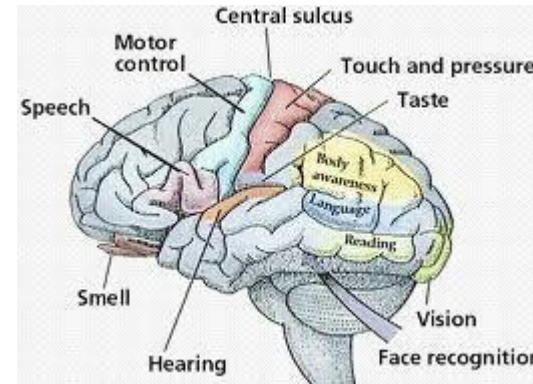
Tipični koraci u primeni DL



↑
Izdvajanje osobina kao ulaza (*feature extraction*) u DL model je automatizovano, a ne ručna operacija.

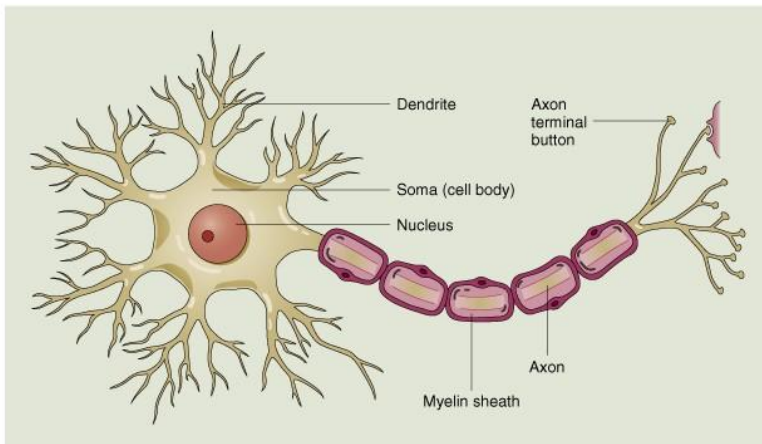
Biološki neuroni

- čine ih:
 - **dendriti** - ulazni kanali el.-hem. signala
 - **soma** - telo ćelije, stiču se ulazi sa dendrita
 - **akson** - jedan, izlazni kanal
- ćelija ima 1000-100000 ulaza i samo 1 izlaz
- **sinapsa** je oblast elektro-hemijskog kontakta među neuronima (pobudna ili inhibitorna)
- način rada: ulazni signali polarizuju somu i kada polarizacija pređe prag, neuron se prazni odašiljući el.-hem. impulse duž aksona



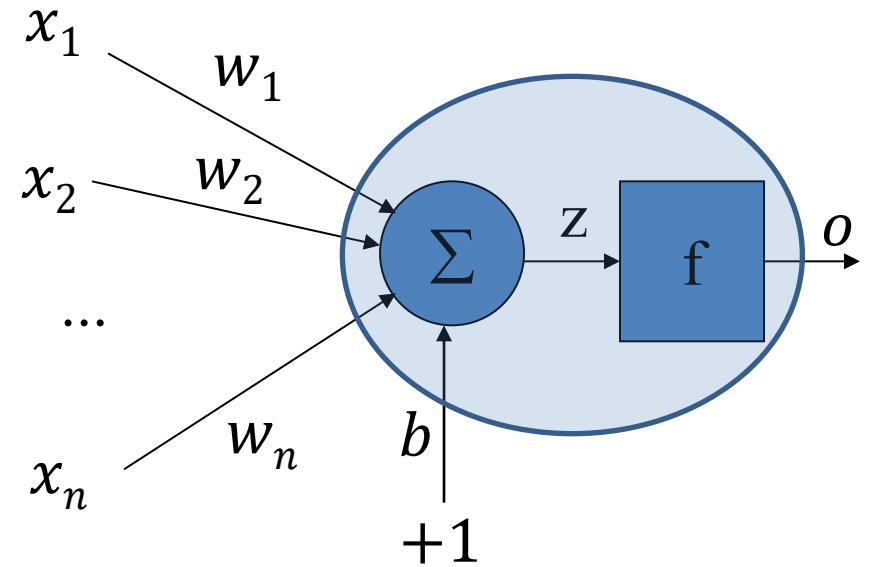
Neke generalije ljudskog mozga

- masa 2% mase tela
- potrošnja kiseonika $> 20\%$ od potrošnje tela
- snaga 20 W
- broj neurona 10^{11}
- broj sinapsi (veza) po neuronu $10^3 - 10^5$
- ukupan broj sinapsi 10^{15}
- maksimalna dužina prenosnog signala $> 1m$
- dimenzija kore 2000 cm^2 , debljina 2 – 3 mm
- kolona je najmanja funkcionalna jedinica sa oko 4000 neurona
- radi: svest, razmišljanje, emocije, ...



Model veštačkog neurona

- Neuron je osnovni procesni element VNM
- Sadrži:
 - ulaze $x_i, i = 1, 2, \dots, n$
 - sinapse (težinski, ponderišući faktori ulaza) w_i i prag b
 - stanje aktivacije z
 - Jedan izlaz o iz aktivacione funkcije $f(z)$
$$o = f(z) = f(\sum_i w_i x_i + b)$$
- Podesivi parametri su: $w_1, w_2, \dots, w_n$ i b (parametri koji se menjaju obučavanjem)



Sličnosti sa biološkim neuronima:

- samo inspiracija potiče iz biologije
- način povezivanja ne odgovara pravom, ali je sličan
- funkcija je slična pravim: učenje na osnovu iskustva, generalizacija i zaključivanje, apstrakcija,...

Tipovi aktivacione funkcije neurona

a) linearna $f(z) = a \cdot z, a = \text{const}$

b) pragovska funkcija $f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$

c) semi-linearna $f(z) = \begin{cases} a, & z \geq q \\ \frac{a}{q} \cdot z, & -q \leq z \leq q \\ -a, & z \leq -q \end{cases}$

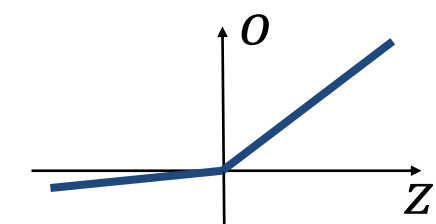
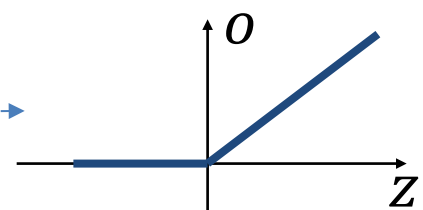
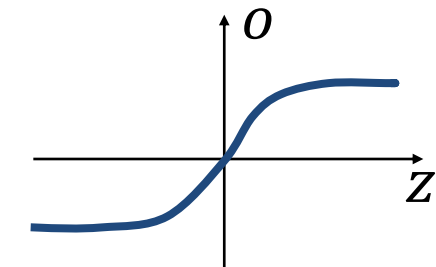
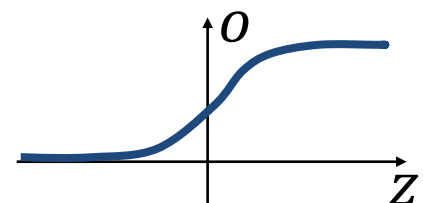
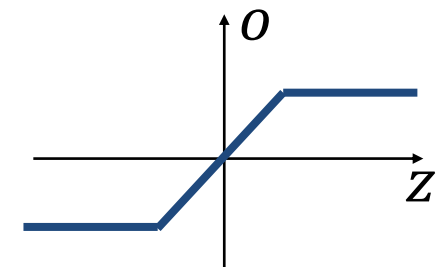
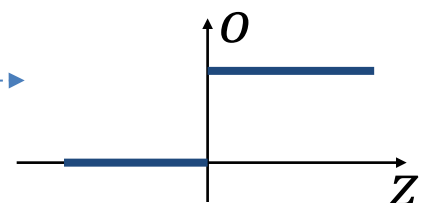
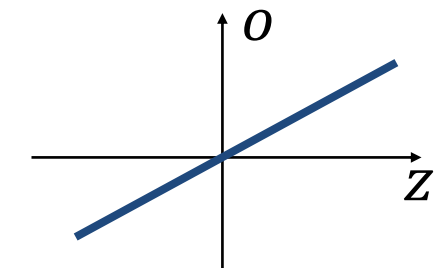
d) log-sigmoidalna $f(z) = \frac{1}{1+e^{-z}}$

e) hiperbolični tangens (tanh) $f(z) = \frac{2}{1+e^{-z}} - 1$

f) ReLU (*Rectified Linear Unit*) $f(z) = \max(0, z)$

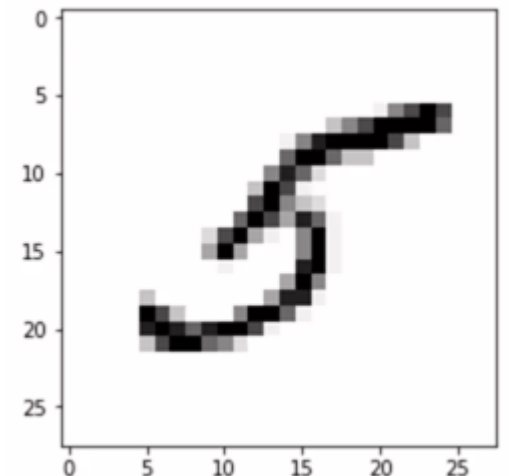
g) Leaky ReLU $f(z) = \max(0.1z, z)$

...



Neuronska mreža

- Generalno, VNM se sastoji od **međusobno povezanih neurona**
 - *motivacija*: značajna sposobnost mreže da „računa“ kada ima dovoljno velik broj neurona
- Arhitektura mreže (broj ulaza, broj izlaza, tipovi neurona i njihove međusobne veze) se bira prema problemu koji se rešava (čitaj „modelu koji se pravi“)
 - Izlazi mreže se izračunaju na osnovu postavljenih ulaza, stanja u mreži (kod nekih tipova) i svakako od vrednosti podesivih parametara neurona (w_i, b)
- Tipovi ulaznih podataka:
 - Sekvencijalni podaci – gde je redosled bitan
 - Tekst, fragmenti zvuka, vremenske serije (merenja), ...
 - Slike i 2D podaci – tipično matrice piksela
 - Tabelarni podaci – logični i sistematsko uređeni podaci (kolona tabele je neka od osobina...)
- Slično je sa izlaznim podacima (ali tipično jednostavnije)
- Da bi VNM bila upotrebljiva **mora se obučiti** – parametri neurona se podese



Obučavanje VNM

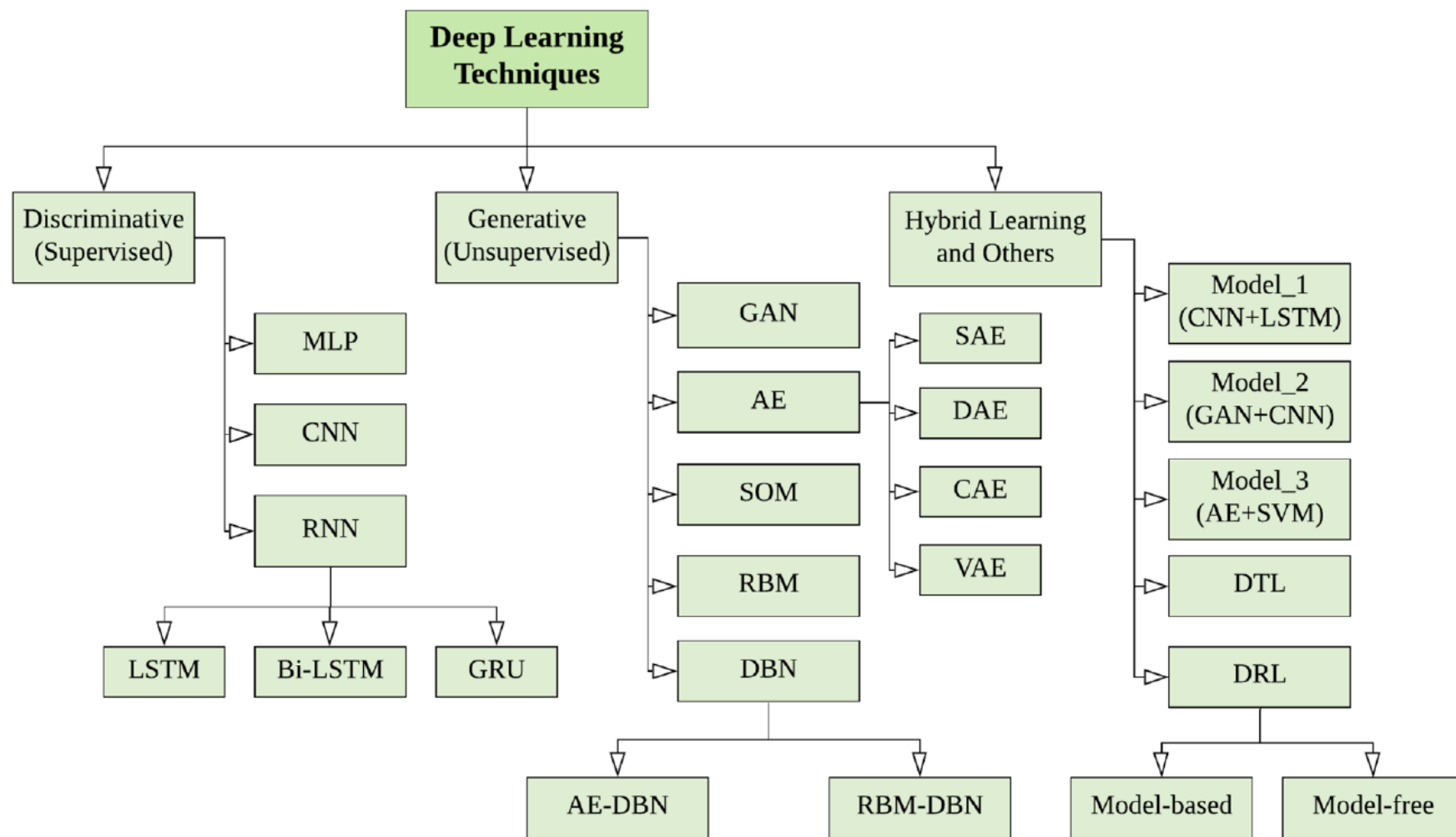
- VNM uči na osnovu serije primera (uzoraka), koji se nazivaju **obučavajući skup**
- Obučavanjem se menjaju parametri neurona (w_i, b)
- Načini obučavanja su:
 - **sa učiteljem** ili supervizorsko (*supervised, diskriminative*)
 - zahteva označavanje ulaznih podataka – za date ulaze se znaju željeni izlazi
 - obučavajući skup čini niz parova {ulazi, željeni izlazi}
 - u procesu obuke se doteruju parametri modela (mreže) dok se na izlazu ne pojavi vrednost željenih izlaza
 - **bez učitelja** ili nesupervizorsko (*unsupervised, generative*)
 - ulazni podaci nisu označeni i od modela se očekuje da „sam“ klasifikuje podatke, detektuje, ...
 - hibridno ili mešovito
 - složenije arhitekture imaju delova obučavane sa i bez učitelja

Postoji i *reinforcement* ili pojačano učenje, kao posebna tehnika kod sekvencijalnog donošenja odluka. Uvedeni su koncepti okruženja i agenta (DNN) gde agent ostvaruje sekvencu akcija nad okruženjem i utiče na njegovo stanje, a biva nagrađivan/kažnjavao za dobra/loša stanja.

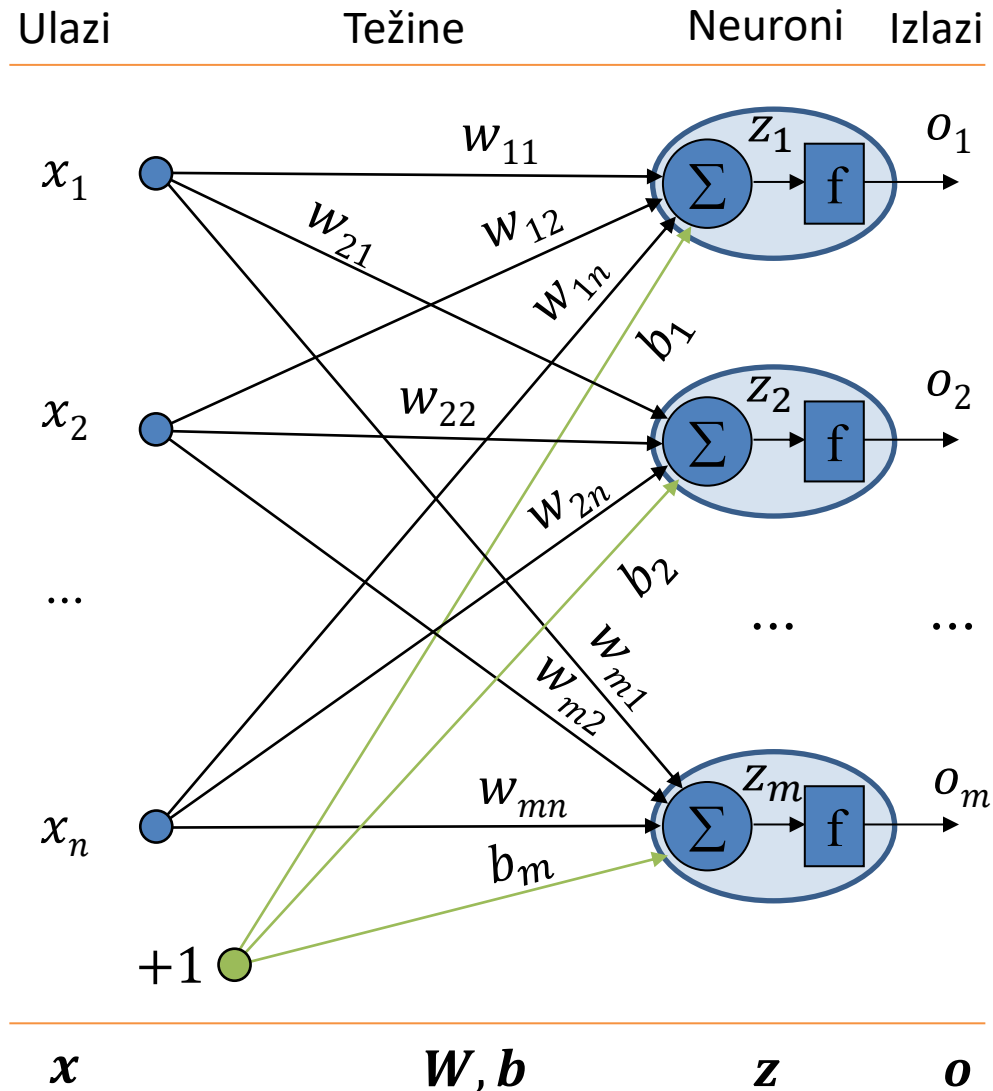
Pregled osnovnih tipova VNM – arhitektura

- Arhitekture VNM gde se koristi DL
- Često su nazvane *Deep Neural Network* (DNN)

(posmatramo samo MLP, a spominjemo samo CNN i RNN)



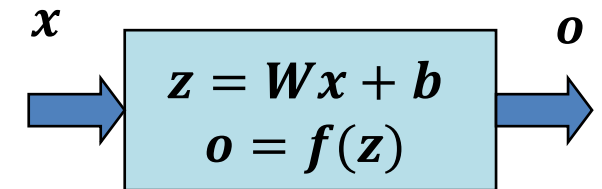
Sloj neurona (tipa *fully connected*)



Podaci su organizovani u vektore i matrice

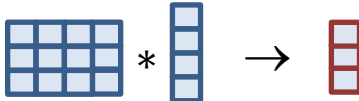
fully connected – svaki neuron se povezuje sa svim ulazima (ili kod MLP sa neuronima prethodnog sloja)

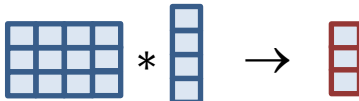
Ovde je pogodna vektorska obrada podataka (rad sa matricama i vektorima)



Napomene za računanje izlaza sloja neurona

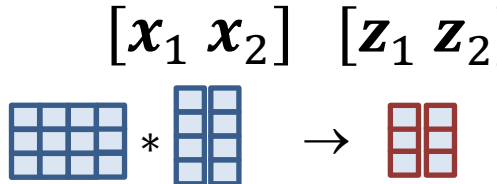
- k -ti neuron u sloju ima parametre:
 - vektor težina $w_{k1} \dots w_{km}$ koje se smeštaju u matricu $\mathbf{W}$ u k -tu vrstu
 - vrednost praga b_k u vektoru $\mathbf{b}$
- Broj kolona $\mathbf{W}$ je jednak broju ulaza u sloj
- Stanje aktivacije svih neurona u sloju je $\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}$, gde je $\mathbf{x}$ vektor ulaza
- Izlaz iz sloja je f-ja aktivacije $\mathbf{o} = \mathbf{f}(\mathbf{z})$


$$\mathbf{W} \mathbf{x}_1 \rightarrow \mathbf{z}_1$$


$$\mathbf{W} \mathbf{x}_2 \rightarrow \mathbf{z}_2$$

Pogodno je izračunati izlaze za više različitih ulaza u sloj i tada se:

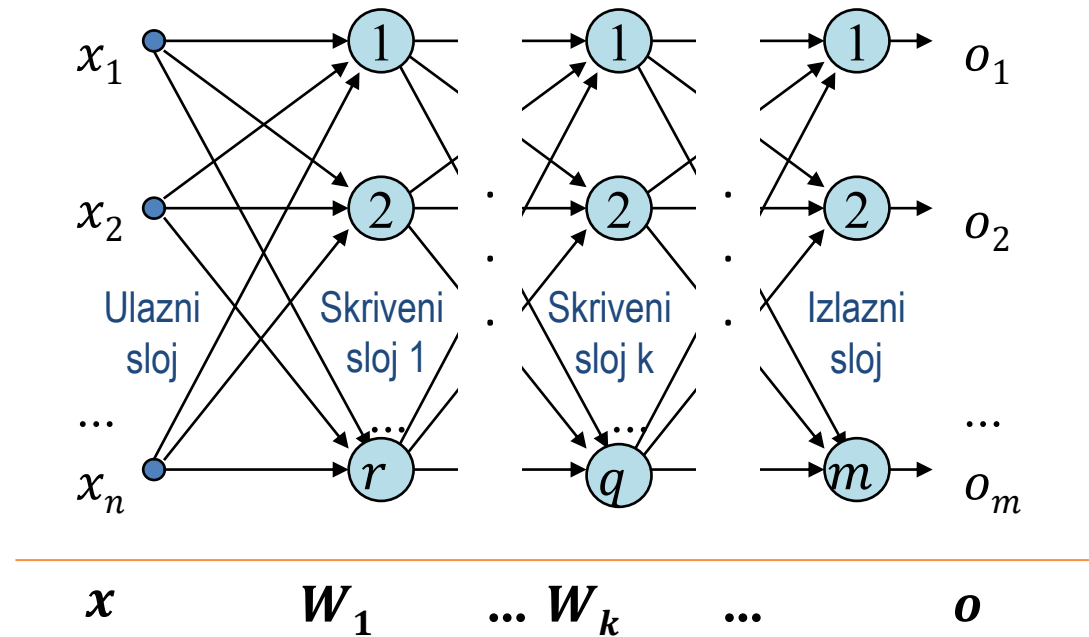
- Pojedinačni vektori ulaza $\mathbf{x}_i$ kao vektori kolone slože u matricu ulaza $\mathbf{X}$
- Izračuna se $\mathbf{Z} = \mathbf{W}\mathbf{X} + \mathbf{b}$ gde se vektor $\mathbf{b}$ sabere sa svakom kolonom $\mathbf{W}\mathbf{X}$ (primetiti da je $\mathbf{Z}$ dimenzije (broj neurona, broj vektora ulaza))
- f-ja aktivacije se primeni na svaku vrednost iz $\mathbf{Z}$, i dobije se matrica $\mathbf{O} = \mathbf{f}(\mathbf{Z})$ dimenzije (broj neurona, broj vektora ulaza)


$$\mathbf{W} \mathbf{X} \rightarrow \mathbf{Z}$$

(ilustracija izostavlja vektor $\mathbf{b}$)

Višeslojni perceptron – *Multi-layer Perceptron* (MLP)

- ili višeslojna *feed-forward* VNM
- VNM sa neuronima organizovanim u slojeve
 - ulazi u VNM se nalaze u ulaznom sloju (niz vrednosti, što nisu neuroni)
 - sloj (niz) neurona istog tipa aktivacione funkcije povezan sa svim ulazima ili izlazima neurona iz prethodnog sloja
 - može ih biti više - **skriveni sloj(evi)**
 - brojevi neurona po slojevima variraju
 - izlazni sloj je poslednji sloj neurona
 - **sadrži izlaze VNM**
- Kod izračunavanja odziva VNM signali propagiraju isključivo od ulaza ka izlazima (prvo se računa odziv neurona prvog skrivenog sloja, pa izlazi drugog skrivenog sloja, ... i na kraju izlazi poslednjeg – izlaznog sloja).



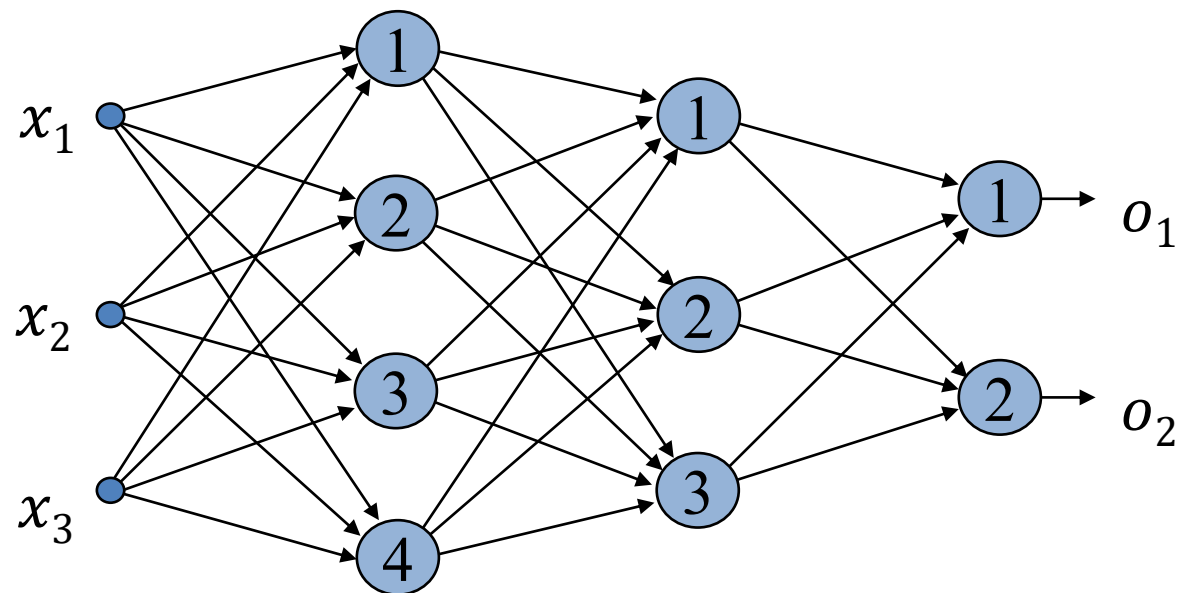
$$o = f_k(W_k \dots f_1(W_1 x + b_1))$$

feed-forward == signali propagiraju samo unapred
Kod obučene mreže parametri W i b su poznati i izlazi mreže zavise samo od ulaza

Primer *feed-forward* VNM sa dva skrivena sloja

Mreža ima:

- 3 ulaza
- 2 izlaza
- 2 skrivena sloja neurona sa po 4 i 3 neurona
- Ukupan broj neurona je 9 ($4 + 3 + 2$)
- Ukupan broj podesivih parametara je 39
 - Prvi sloj $4 \times 3 + 4 = 16$
 - Drugi sloj $3 \times 4 + 3 = 15$
 - Treći sloj / izlazni $2 \times 3 + 2 = 8$

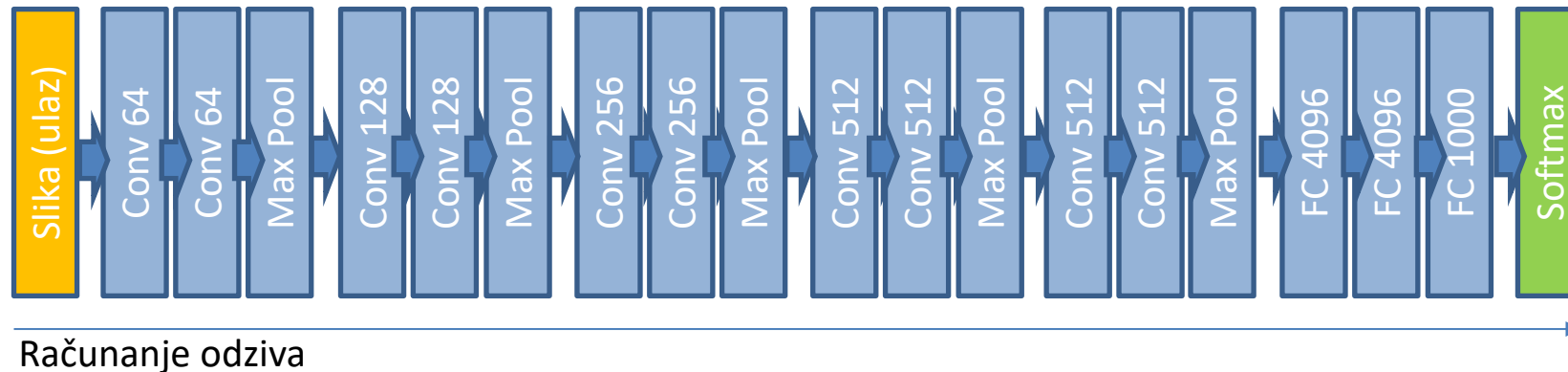


Napomena: svaki neuron ima parametar prag b , ali on nije nacrtan na slici.

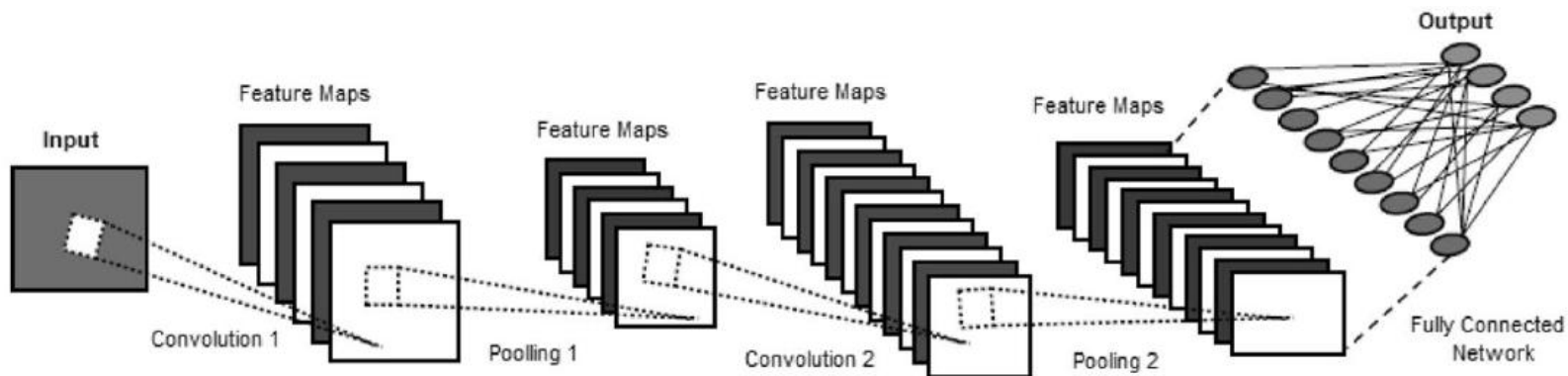
Konvoluciona neuronska mreža – *Convolutional Neural Network (CNN)*

- Višeslojna *feed-forward* VNM sa kombinovanim tipovima slojeva: *fully connected, convolutional, normalisation, max pool, ...*, koja se obučava „sa učiteljem“
- Tipična upotreba je u računarskoj viziji (computer vision)

Primer 1



Primer 2
šematski
prikaz



Rekurentne VNM – *Recurrent Neural Network* (RNN)

- Radi sa sekvencijalnim podacima, poput vremenskih serija
- Koristi stanje h_{t-1} iz prethodnog koraka ($t - 1$ trenutka) da odredi stanje h_t u tekućem koraku (t trenutak)
- određuje izlaz na osnovu ulaza u_t i vektora stanja h_t (ima internu memoriju) koji zavisi ranijih ulaza
- Postoji više podtipova RNN, npr.:
 - Long Short-Term Memory* (LSTM) sadrži ćelije sa po 3 kapije:
 - kapija zaboravljanja (težine W_{hh}) – određuje koje informacije iz prethodne ćelije se pamte, a koje će biti uklonjene jer više nisu korisne,
 - ulazna kapija (težine W_{xh}) – određuju koje informacije treba da uđu u stanje ćelije
 - izlazna kapija (težine W_{hy}) – određuje i kontroliše izlaze.

$$h_t = f_w(h_{t-1}, x_t) = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

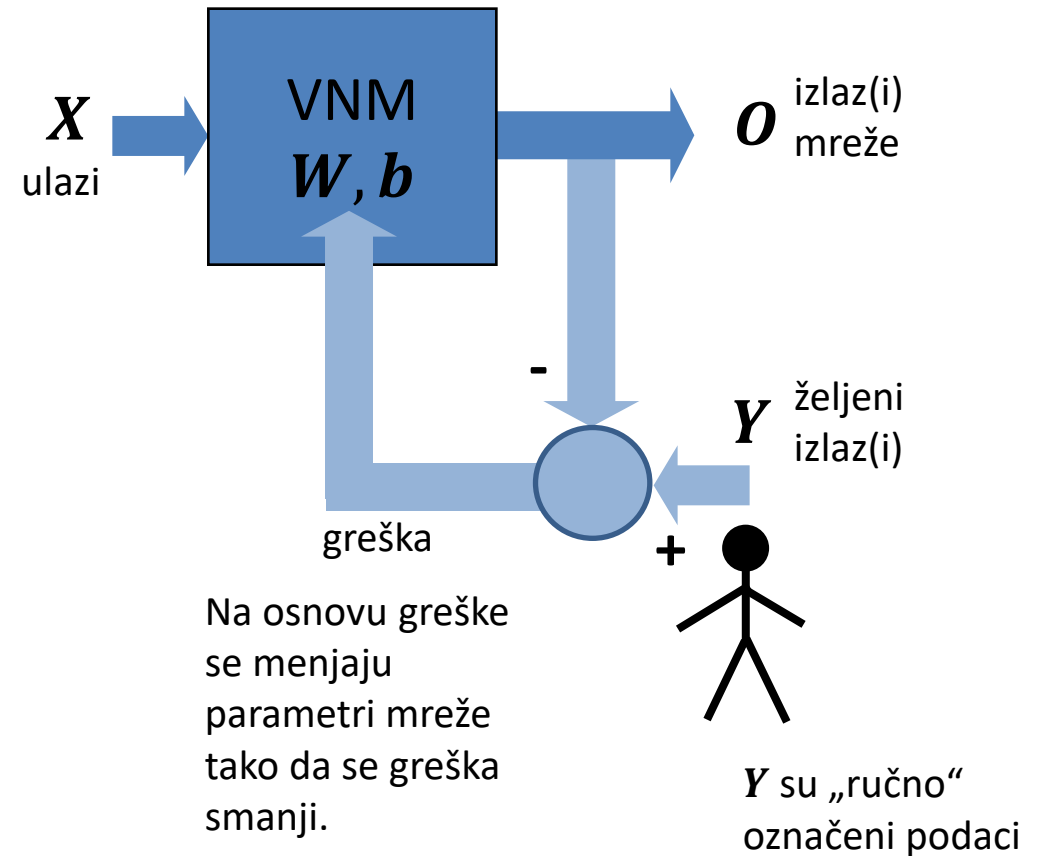
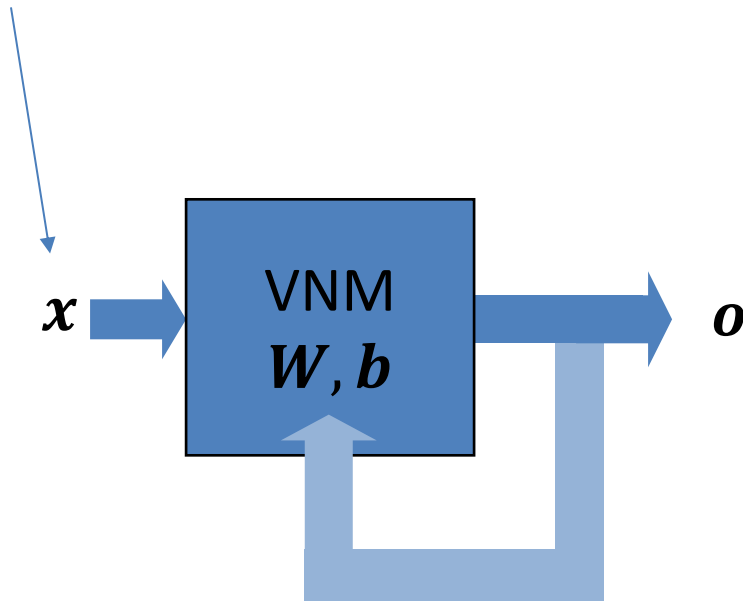


Primer rekurentne VNM

Modeli obučavanja VNM (sada detaljnije)

Modeli obučavanja su:

- **supervizorsko** (sa učiteljem)
 - postoji obučavajući skup, tj. parovi: {ulazi, željeni-izlaz(i)}
- **nesupervizorsko** (bez učitelja)
 - postoje samo ulazni podaci

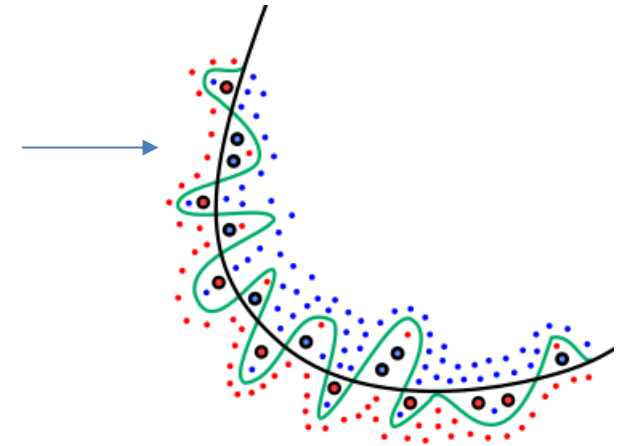


Supervizorsko obučavanje

- Cilj: Podesiti parametre mreže – težine ($\mathbf{W}$ i $\mathbf{b}$) tako da se minimizuje funkcija cilja $\mathcal{J}(\mathbf{W}, \mathbf{b})$
- Funkcija cilja $\mathcal{J}$ (*Loss function*) tipično predstavlja razliku (rastojanje) željenog i ostvarenog izlaza mreže
 - Na osnovu funkcije cilja, tipa sloja i aktivacionih funkcija definiše se pravilo (algoritam) korekcije parametara mreže $\Delta w, \Delta b$
- Za obučavanje se upotrebljava **obučavajući skup** – tj. podaci za obučavanje
- Obučavanje se sprovodi **iterativno** – u više ciklusa (**epoha**) $k = 0, 1, 2, \dots$
 - Jedan ciklus obučavanja je jedan prolaz kroz (ceo ili deo) obučavajućeg skupa nakon čega se ažuriraju parametri
$$w_{k+1} = w_k + \Delta w$$
$$b_{k+1} = b_k + \Delta b$$

Supervizorsko obučavanje (2)

- Koristi se obučavajući skup sačinjen od P parova (ulaz $\mathbf{x}_i$ – željeni izlaz $\mathbf{y}_i$):
$$\{(\mathbf{x}_1, \mathbf{y}_1), (\mathbf{x}_2, \mathbf{y}_2), \dots (\mathbf{x}_P, \mathbf{y}_P)\}$$
- Obuka traje dok se izlazi mreže $\mathbf{o}_i$ “ne poklope” sa željenim izlazima $\mathbf{y}_i, i = 1, 2, \dots P$ i to često zahteva veliki broj epoha
- Model zasnovan na VNM treba da je **prediktivno valjan** i **loše je kada se mreža „pretrenira“**, tj. nauči samo podatke na osnovu kojih se trenirala.
- Stoga, obučavajući skup se deli na 3 dela:
 1. Podskup za obuku – koristi se za promene parametara mreže
 2. Podskup za kontrolu (kraja obuke)
 3. Podskup za testiranje obučene mreže (ne koristi se u obuci)



Tokom obuke parametri mreže se isključivo menjaju na osnovu podskupa za obuku i tada se vrednost funkcije cilja tipično smanjuje iz epohe u epohu. Teorijski $\mathcal{J} \rightarrow 0$, ali to nije poželjno i „pretreniranost“ se izbegava tako što se računa $\mathcal{J}_2$ nad drugim podskupom obučavajućeg skupa (na osnovu koga se ne menjaju parametri mreže). Tokom obuke $\mathcal{J}_2$ takođe tipično opada, ali kada počne da raste treba prestati sa obukom.

Funkcija cilja – Kriterijum optimalnosti

- Funkcija cilja $\mathcal{J}$ je ukupna greška računata po svim elementima obučavajućeg skupa:

$$\mathcal{J} = \sum_{p=1}^P E_p$$

- Greška E_p predstavlja odstupanje jednog elementa obučavajućeg skupa (suma u donjoj formuli se odnosi na sve izlaze modela):

$$E_p = \frac{1}{2} \sum_{k=1}^m (o_{pk} - y_{pk})^2 = \frac{1}{2} (\mathbf{o}_p - \mathbf{y}_p)^T (\mathbf{o}_p - \mathbf{y}_p)$$

- Kako izlaz mreže, a time i funkcija cilja $\mathcal{J}$ zavise od parametara mreže

$$\mathcal{J} = \mathcal{J}(W, b)$$

optimalne vrednosti parametara se mogu izračunati iz funkcije cilja traženjem

$$\min_{W, b} \mathcal{J}$$

Tako se dobija obučena mreža.

Obučavanje VNM (uopšteno, ML) je minimizacija ukupne greške, tj. funkcije cilja $\mathcal{J}$

Obučavanje VNM: *Back-propagation* algoritam (BP)

- *Back-propagation* (BP) je najčešće upotrebljavan algoritam obuke VNM, pogotovo za MLP
- Koristi gradijentni algoritam za minimizaciju funkcije cilja: $\min_{w,b} \mathcal{J}$

$$\Delta w = -\eta \frac{\partial \mathcal{J}}{\partial w}, \quad \Delta b = -\eta \frac{\partial \mathcal{J}}{\partial b}$$

konstanta $0 < \eta < 1$ je koeficijent brzine obučavanja (*learning rate*)

- Za računanje gradijenata primenjuje se računanje izvoda složene funkcije po promenljivim parametrima
 - Proces računanja polazi od izlaza mreže i kreće se ka ulazu

Kako je

$$\frac{\partial \mathcal{J}}{\partial w} = \sum_{p=1}^P \frac{\partial E_p}{\partial w}$$

treba odrediti

$$\frac{\partial E_p}{\partial w} = ?$$

i

Kako je

$$\frac{\partial \mathcal{J}}{\partial b} = \sum_{p=1}^P \frac{\partial E_p}{\partial b}$$

treba odrediti

$$\frac{\partial E_p}{\partial b} = ?$$

BP: Promena težina u izlaznom sloju

? $\frac{\partial E_p}{\partial W} = \frac{\partial E_p}{\partial f} \frac{\partial f}{\partial z} \frac{\partial z}{\partial W}$

? $\frac{\partial E_p}{\partial b} = \frac{\partial E_p}{\partial f} \frac{\partial f}{\partial z} \frac{\partial z}{\partial b}$

Ovde težina pripada neuronu od čijeg izlaza direktno zavisi greška.

$$E_p = \frac{1}{2} (o - y)^T (o - y), \quad o = f(z), \quad z = Wx + b$$

$$\frac{\partial E_p}{\partial f} = o - y$$

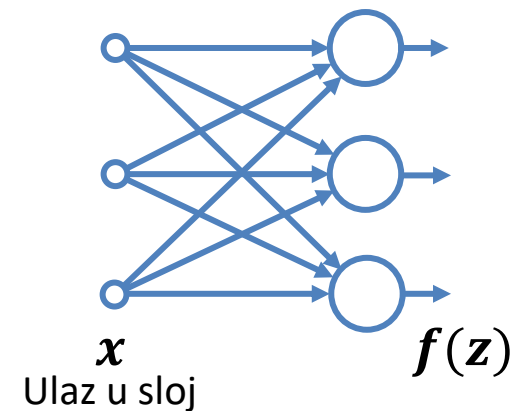
$$\frac{\partial f}{\partial z} = \frac{2e^{-z}}{(1 + e^{-z})^2} = \frac{1}{2}(1 - o^2)$$

$$\frac{\partial z}{\partial W} = x^T$$

$$\frac{\partial z}{\partial b} = 1$$

$$f(z) = \tanh(z) = \frac{2}{1 + e^{-z}} - 1$$

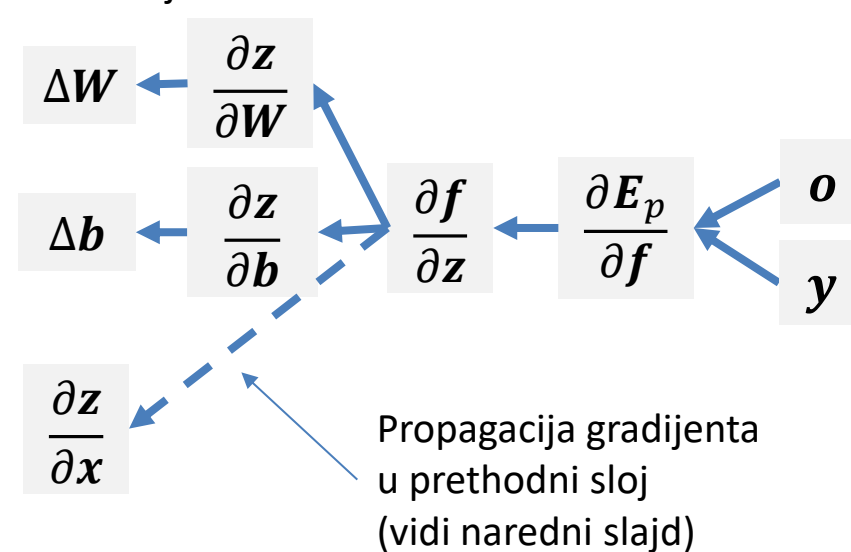
Specifično za izabranu aktivacionu funkciju neurona



Računanje unapred:



Računanje unazad:



BP: Promena težina u skrivenim slojevima

$$E_p = \frac{1}{2} (f(z) - y)^T (f(z) - y)$$

$$z = W o + b$$

Ovde težina pripada neuronu od čijeg izlaza indirektno zavisi greška.

E_p greška na izlazu

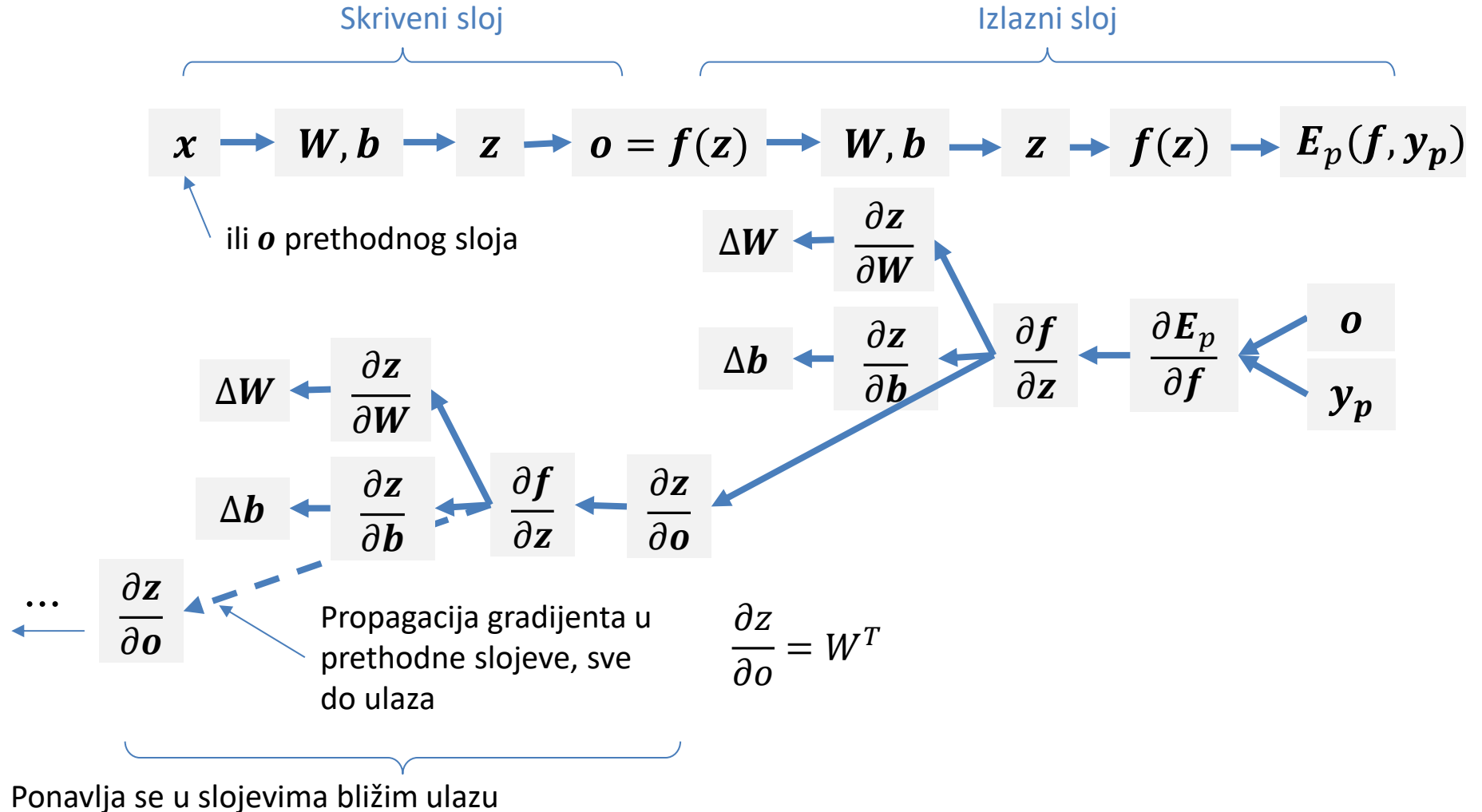
$$\frac{\partial E_p}{\partial W} = \frac{\partial E_p}{\partial o} \frac{\partial o}{\partial W}$$

o izlaz neurona u dubini mreže

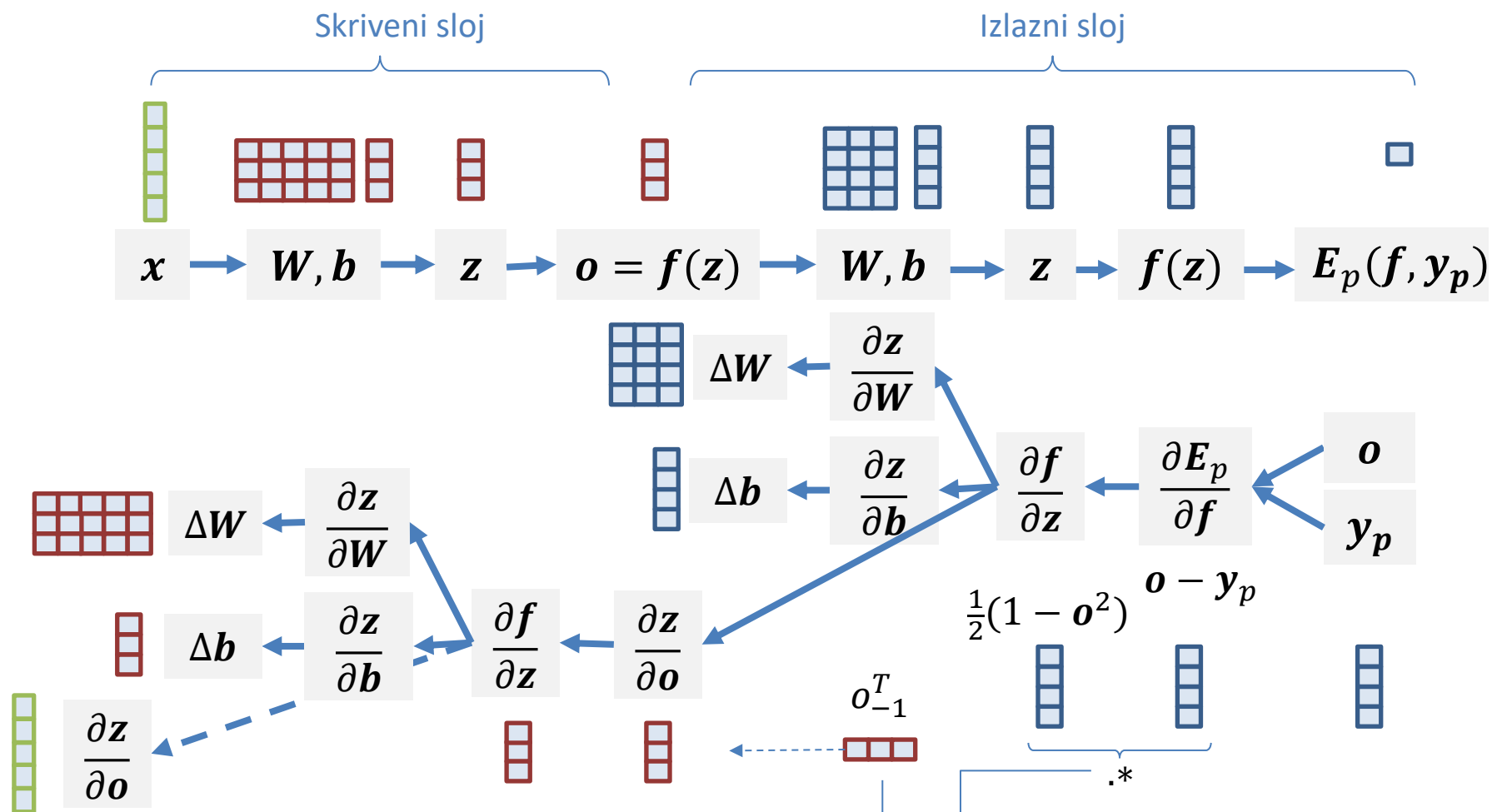
W težina u neuronu

$$\frac{\partial E_p}{\partial b} = \frac{\partial E_p}{\partial o} \frac{\partial o}{\partial b}$$

$$\frac{\partial o}{\partial W} = \frac{\partial o}{\partial f} \frac{\partial f}{\partial z} \frac{\partial z}{\partial W} \quad \leftarrow \text{Nadovezuje se} \quad \frac{\partial E_p}{\partial o} = \frac{\partial E_p}{\partial f} \frac{\partial f}{\partial z} \frac{\partial z}{\partial o}$$



Posmatra se samo jedan element obučavajućeg skupa



$$\frac{\partial E_p}{\partial o} = W^T * \left(\frac{1}{2}(1 - o.*o) .* (o - y_p) \right) = \text{red vector} = \text{blue matrix} * \text{blue vector}$$

$$\Delta W = -\eta \left(\frac{1}{2}(1 - o.*o) .* (o - y_p) \right) * o_{-1}^T = \text{blue matrix} = \text{blue vector} * \text{red vector}$$

$$\begin{aligned} \frac{\partial E_p}{\partial f} &= o - y \\ \frac{\partial f}{\partial z} &= \frac{1}{2}(1 - o^2) \\ \frac{\partial z}{\partial z} &= x^T = o_{-1}^T \\ \frac{\partial z}{\partial o_{-1}} &= W^T \end{aligned}$$

o je izlaz sloja, a o_{-1} je njegov ulaz, tj. izlaz prethodnog sloja.

Posmatramo samo promenu W , a za b je lakše.

Vektorizacija proračuna

Do sada su: težine $\mathbf{W}$ posmatrane kao matrica za sve neurone u sloju, a ulazi $\mathbf{x}$, izlazi $\mathbf{o}$, izlazi $\mathbf{o}_{-1}$ i očekivani izlazi $\mathbf{y}_p$ kao vektori.

Za svaki elemenat obučavajućeg skupa:

$$\Delta \mathbf{W}_p = -\eta \left(\frac{1}{2} (1 - \mathbf{o} * \mathbf{o}) * (\mathbf{o} - \mathbf{y}_p) \right) * \mathbf{o}_{-1}^T = \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} = \begin{array}{|c|} \hline \square \\ \hline \square \\ \hline \square \\ \hline \end{array} * \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \end{array}$$

Posle cele epohe (težine se ažuriraju za svaki elemenat obučavajućeg skupa):

$$\Delta \mathbf{W} = \sum_p \Delta \mathbf{W}_p$$

$$\Delta w = -\eta \frac{\partial \mathcal{J}}{\partial w}$$

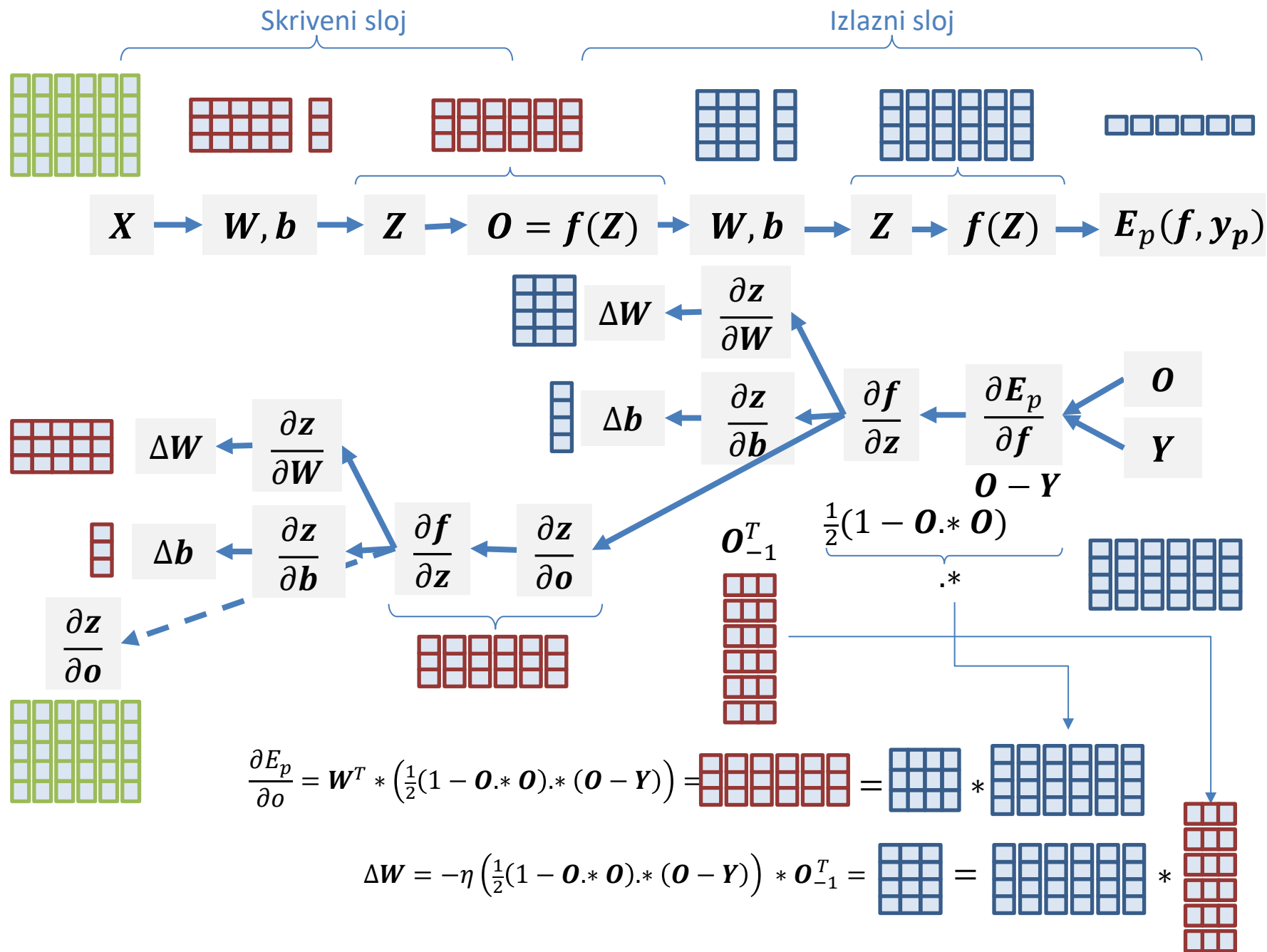
$$\frac{\partial \mathcal{J}}{\partial w} = \sum_{p=1}^P \frac{\partial E_p}{\partial w}$$

Prethodna suma se realizuje množenjem matrica (gde je uzet ceo obučavajući skup):

$$\Delta \mathbf{W} = -\eta \left(\frac{1}{2} (1 - \mathbf{o} * \mathbf{o}) * (\mathbf{o} - \mathbf{Y}) \right) * \mathbf{O}_{-1}^T = \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} = \begin{array}{|c|c|c|c|c|c|} \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \end{array} * \begin{array}{|c|c|c|c|c|c|} \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \square & \square & \square & \square & \square & \square \\ \hline \end{array} \begin{array}{c} \downarrow p \\ p \end{array}$$

Sada je $\mathbf{Y}$ matrica željenih izlaza (ceo obučavajući skup), a matrice izlaza $\mathbf{O}$ i $\mathbf{O}_{-1}$ su računate za sve elemente obučavajućeg skupa. Pošto se u istom izrazu pojavljuju izlazi sloja neurona, kao i njegovi ulazi, tj. izlazi prethodnog sloja upotrebljene su oznake $\mathbf{O}$ i $\mathbf{O}_{-1}$, respektivno.

Posmatra se
više elemenata
obučavajućeg
skupa



Vidi raniji
komentar za
oznake: O i O_{-1}

BP / Delta pravilo: Promena težina u izlaznom sloju

Neka literatura BP algoritam naziva "Delta pravilom" jer se računa δ na svakom neuronu.

$$\delta_{ok} = -\frac{\partial E_p}{\partial z_k}$$

δ_{ok} je k -ti neuron sloja čiji su izlazi o

Ovde je izvođenje promena težina prikazano u razvijenom obliku (bez upotrebe matričnog računa).

$$\Delta w_{kj} = -\eta \frac{\partial E_p}{\partial w_{kj}}$$

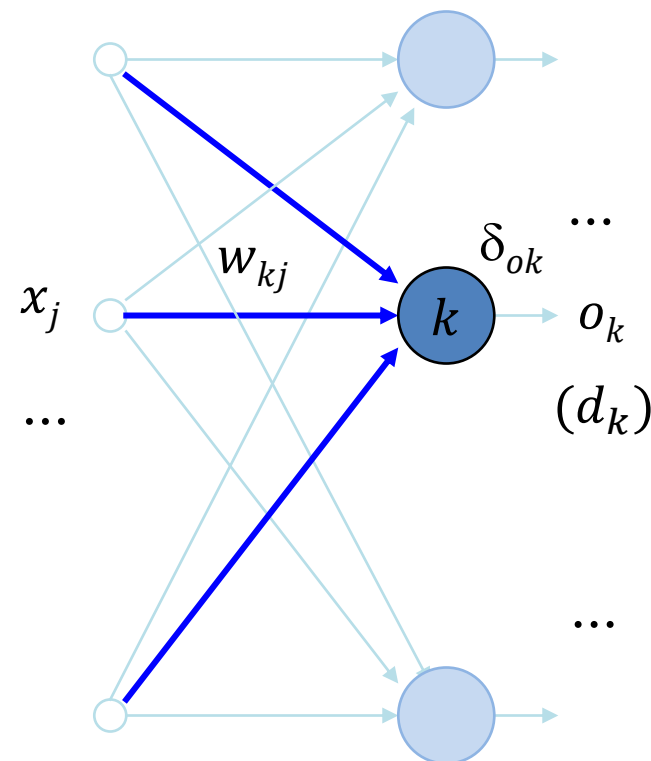
$$\frac{\partial E_p}{\partial w_{kj}} = \frac{\partial E_p}{\partial o_k} \frac{\partial o_k}{\partial w_{kj}} = \underbrace{\frac{\partial E_p}{\partial o_k} \frac{\partial o_k}{\partial z_k}}_{-\delta_{ok}} \frac{\partial z_k}{\partial w_{kj}}$$

$$\frac{\partial E_p}{\partial o_k} = -(d_k - o_k)$$

$$\frac{\partial o_k}{\partial z_k} = \frac{1}{2}(1 - o_k^2) \text{ za } o(z) = \frac{2}{1+e^{-z}} - 1$$

$$\frac{\partial z_k}{\partial w_{kj}} = x_j$$

$$\Delta w_{kj} = \eta (d_k - o_k) \frac{1}{2}(1 - o_k^2) x_j$$



$$\Delta w_{kj} = \eta \delta_{ok} x_j$$

$$\delta_{ok} = \frac{1}{2}(1 - o_k^2)(d_k - o_k)$$

BP / Delta pravilo : Promena težina u skrivenim slojevima

...

Radi jednostanosti težine izlaznog sloja su w , a skrivenog sloja v .

δ_{yj} je j -ti neuron skrivenog sloja čiji su izlazi y

Primetiti da je računanje δ_{yj} za neuron svodi na prenošenje delte iz narednog sloja preko plavih težina $(\sum_{k=1}^m \delta_{ok} w_{kj})$, a koristi se za računanje Δv_{ji} u crvenim težinama.

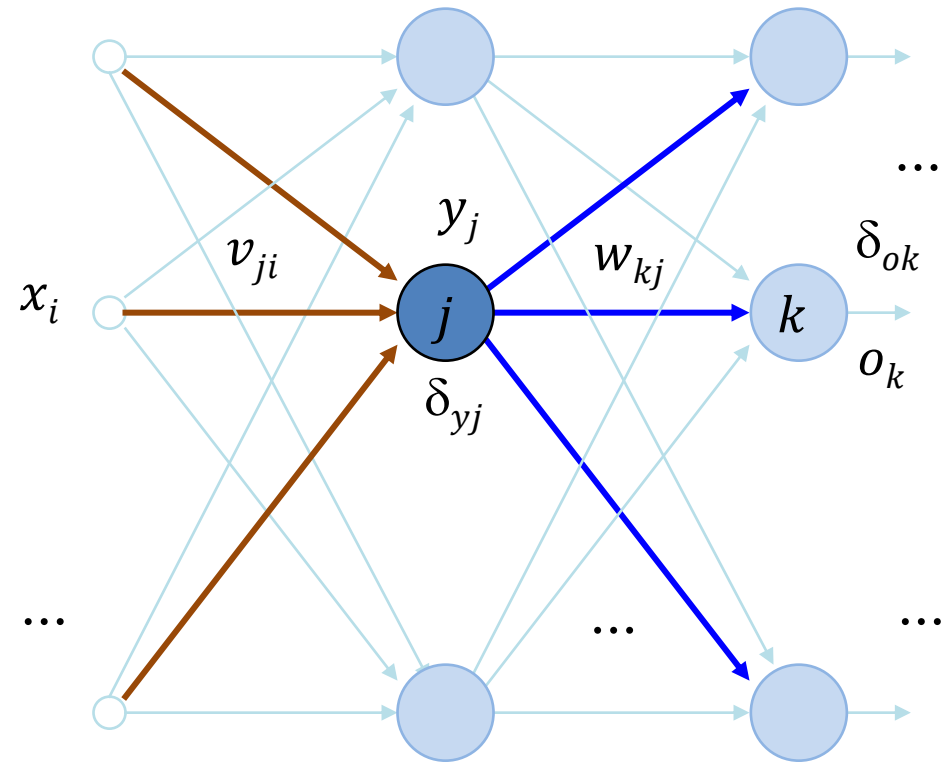
$$\Delta v_{ji} = -\eta \frac{\partial E_p}{\partial v_{ji}}$$

$$\frac{\partial E_p}{\partial v_{ji}} = \frac{\partial E_p}{\partial y_j} \frac{\partial y_j}{\partial v_{ji}}$$

$$\frac{\partial E_p}{\partial y_j} = \sum_k \frac{\partial E_k}{\partial z_k} \frac{\partial z_k}{\partial y_j} = -\sum_k \delta_{ok} w_{kj}$$

$$\frac{\partial y_j}{\partial v_{ji}} = \frac{1}{2}(1 - y_j^2)x_i$$

$$\Delta v_{ji} = \eta \left(\sum_k \delta_{ok} w_{kj} \right) \frac{1}{2} (1 - y_j^2) x_i$$

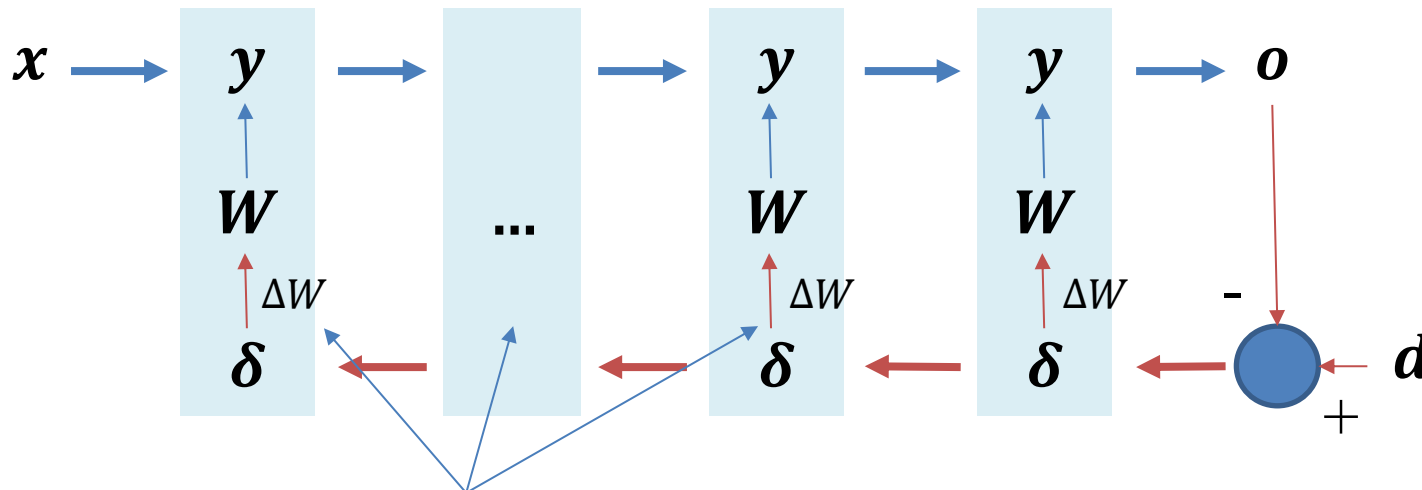


$$\Delta v_{ji} = \eta \delta_{yj} x_i$$

$$\delta_{yj} = \frac{1}{2} (1 - y_j^2) \sum_{k=1}^m \delta_{ok} w_{kj}$$

BP: Promena težina (objedinjeni prikaz)

- Posmatra se VNM sa više slojeva ...
 - Izlazni sloj: $\Delta w_{kj} = \eta \delta_{ok} x_j$
 $\delta_{ok} = \frac{1}{2}(1 - o_k^2)(d_k - o_k)$
 - Skriveni slojevi: $\Delta v_{ji} = \eta \delta_{yj} x_i$
 $\delta_{yj} = \frac{1}{2}(1 - y_j^2) \sum_{k=1}^m \delta_{ok} w_{kj}$
- gde je:
 δ vezano za neuron,
 x su njegovi ulazi
- δ se prvo izračuna za izlazni sloj, a zatim za skrivene slojeve (pomerajući se na ulazima mreže)



Skriveni slojevi ažuriraju težine po formuli za Δv_{ji} , a ovde su te izmene označene sa ΔW zbog uniformnosti.

Koraci BP algoritma i implementacija

Koraci:

1. inicijalizuju se težine $\mathbf{W}$ i $\mathbf{b}$ na slučajne male vrednosti ($\mathbf{b} = \mathbf{0}$)
2. postavi se ulaz $\mathbf{X}$ i izračunaju se izlazi neurona $\mathbf{O}$
(preko izlaza u skrivenim slojevima)
3. Izračunaju se gradijenti $\mathcal{J}$ po parametrima i na osnovu njih: $\Delta\mathbf{W}, \Delta\mathbf{b}$
(gde izvodi propagiraju u nazad - *back-propagation*)
4. koriguju se parametri mreže: $\mathbf{W}, \mathbf{b}$
5. Proveri se da li je mreža obučena (npr. izračuna se greška E_p , tj. ukupna greška $\mathcal{J}$)?
Ako nije nastavlja se novom epohom (od koraka 2)

Implementacija BP:

- Model neuronske mreže sadrži slojeve (niz slojeva).
- Svaki sloj koristi matrice i vektore da modeluje parametre (težine) sloja i računanje vezano za sloj.
- Matrice modeluju obučavajući skup (dimenzije: broj ulaza/izlaza $\times$ broj el. obučavajućeg skupa).
- Računanje po BP algoritmu se vrši za ceo obučavajući skup (ili njegov deo) - cela epoha u jednoj iteraciji.

Ubrzanja konvergencije BP algoritma i drugi algoritmi obuke

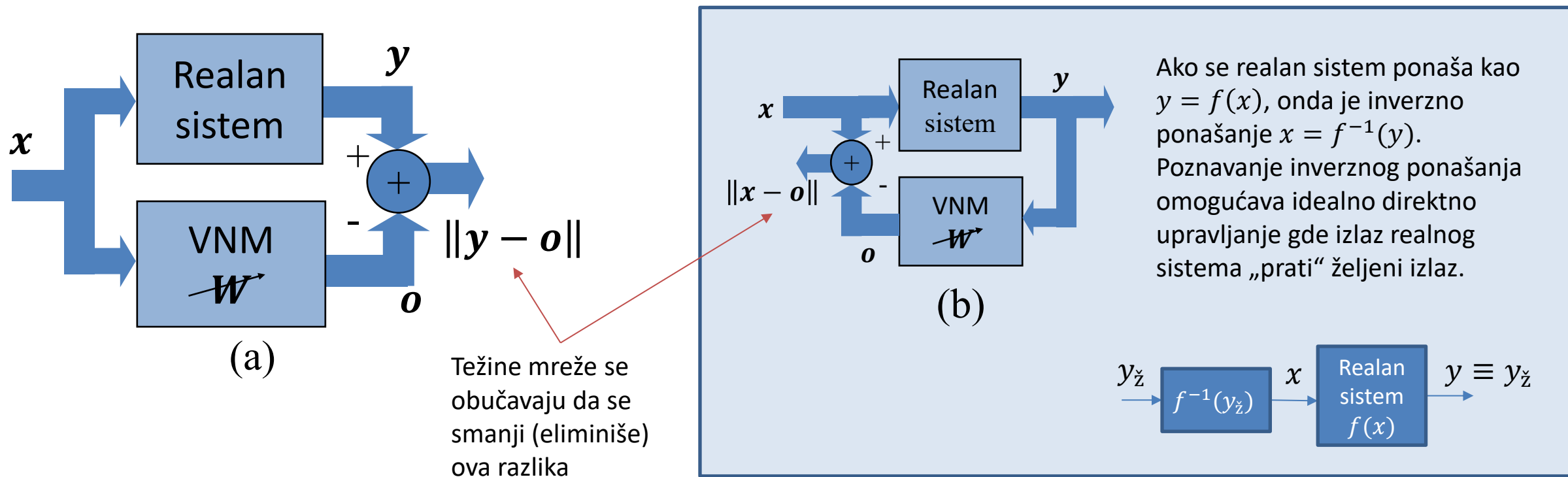
- Dodavanje “momentum člana” tipično ubrza obuku klasičnog BP alg.:

$$\begin{aligned} V_{k+1} &= \gamma V_k + \eta \frac{\partial J}{\partial W} & v_{k+1} &= \gamma v_k + \eta \frac{\partial J}{\partial b} \\ W_{k+1} &= W_k - V_{k+1} & b_{k+1} &= b_k - v_{k+1} \end{aligned}$$

- ovde je k indeks oznaka iteracije, a parametar algoritma γ je “momentum faktor” ($\gamma \approx 0.9$)
- ovo je u teoriji optimizacije varijanta algoritma konjugovanih gradijenata
- Neki drugi algoritmi:
 - *Stochastic Gradient Descent* (SGD)
 - *Limited Memory BFGS* (L-BFGS)
 - *Adaptive Moment Estimation* (Adam)
 - ...

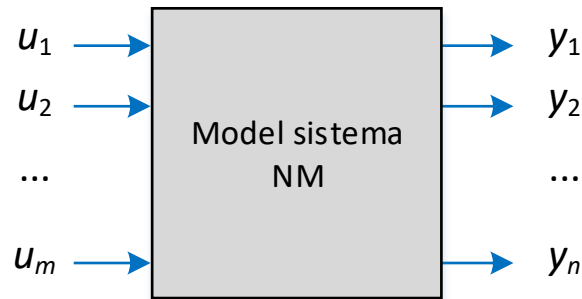
Primena VNM u modeliranju i simulaciji

- Sposobnost NM da (proizvoljno) mapira ulaze na izlaze joj omogućava da modelira (i simulira) ponašanje sistema
- Obučavanje VNM ulazno-izlaznim podacima iz sistema predstavlja postupak identifikacije sistema
- Mogu se vršiti:
 - direktna identifikacija (a) – bitna i česta u upotrebi
 - inverzna identifikacija (b) – retka u upotrebi i nije uvek moguća

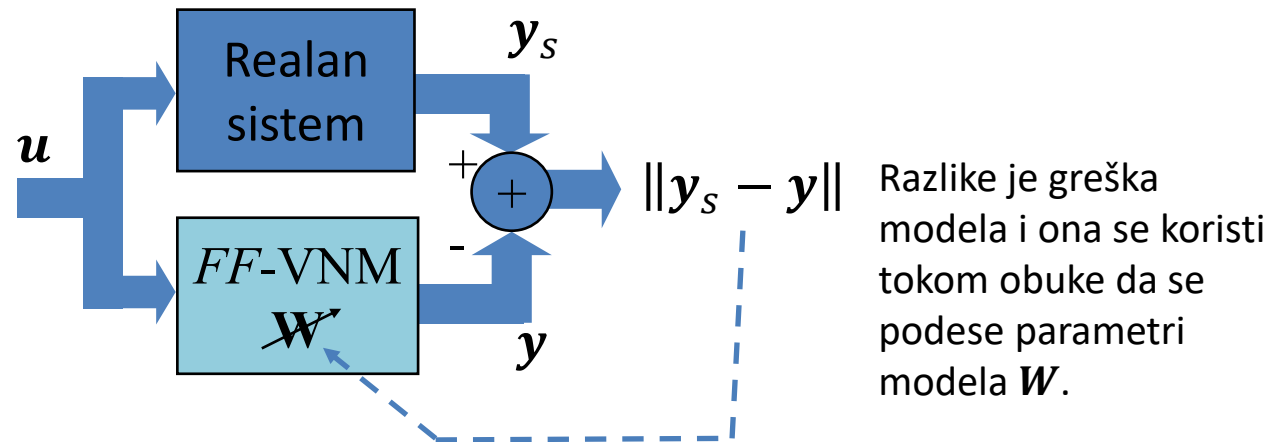


Statički model

- Potrebno je da VNM nauči preslikavanje skupa ulaza na skup izlaza

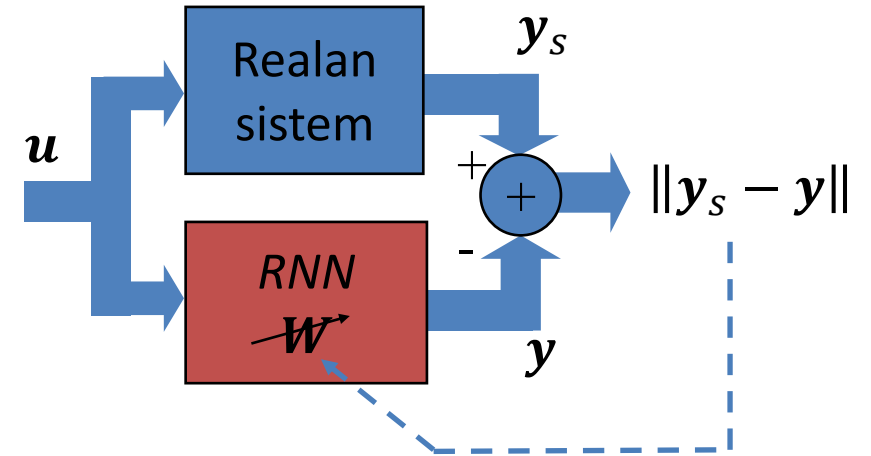


- ovo preslikavanje **ne zavisi od vremena** (od stanja u kome se sistem nalazi) te se model **može realizovati *feed-forward* veštačkom neuronskom mrežom**.



Dinamički model preko RNN

- Preslikavanje ulaza na izlaze **zavisi od vremena** (od stanja u kome se sistem nalazi) te se model **ne može realizovati *feed-forward* neuronskom mrežom**.
- Potrebno je uspostaviti stanje u modelu (jer se njime realizuje memorisanje uticaja ranijih ulaza)
- Dinamički model se može realizovati upotrebom rekurzivnih neuronskih mreža (RNN)
 - izlazi zavise od ulaza i od stanja

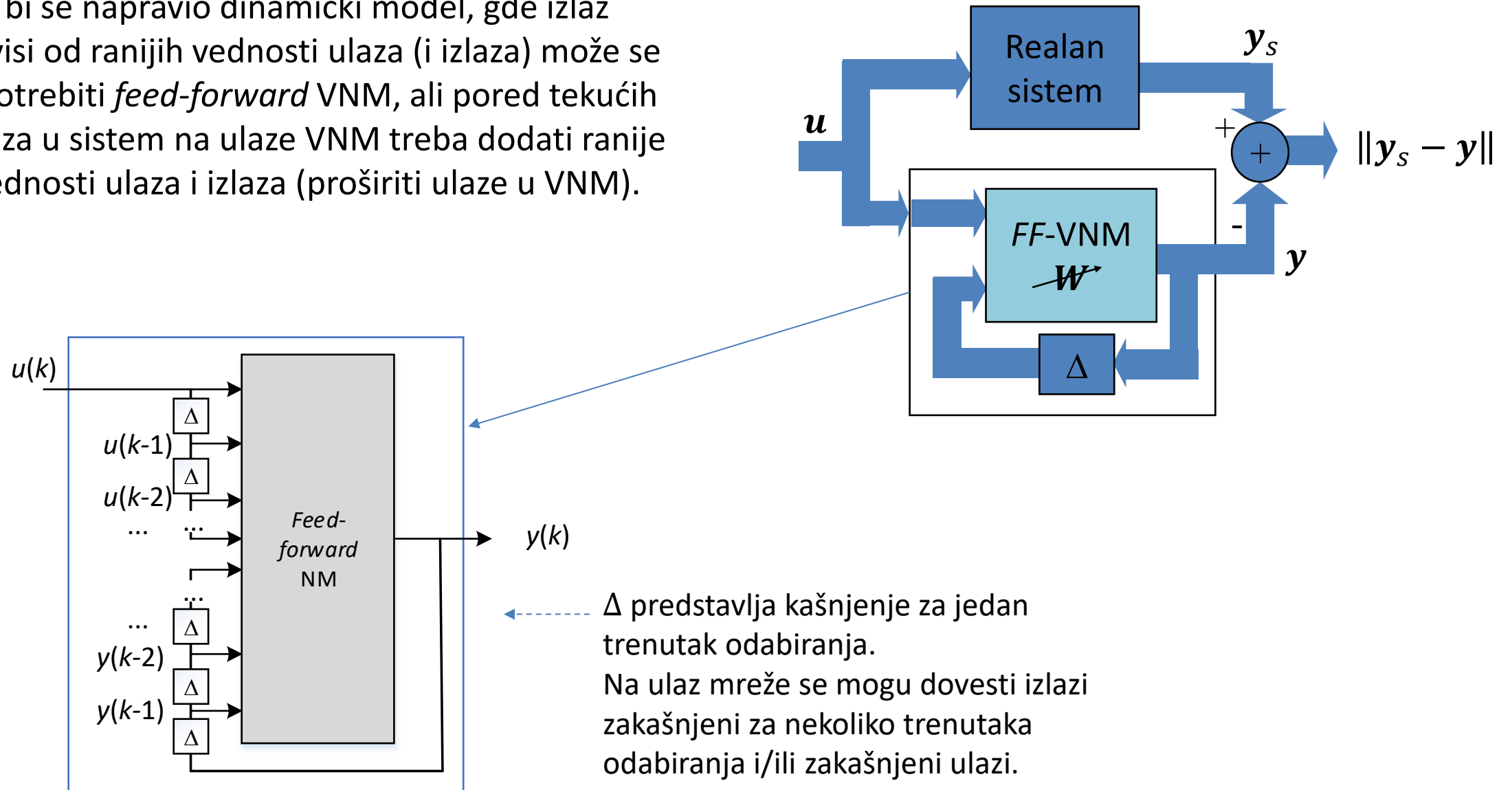


I ovde se razlikom izlaza sistema i modela predstavlja greška modela i ona se koristi tokom obuke da se podese parametri modela W .

Primetiti da je promenjen tip VNM u RNN za razliku od statičkog modela gde je dovoljna *feed-forward* VNM tipa višeslojnog perceptrona.

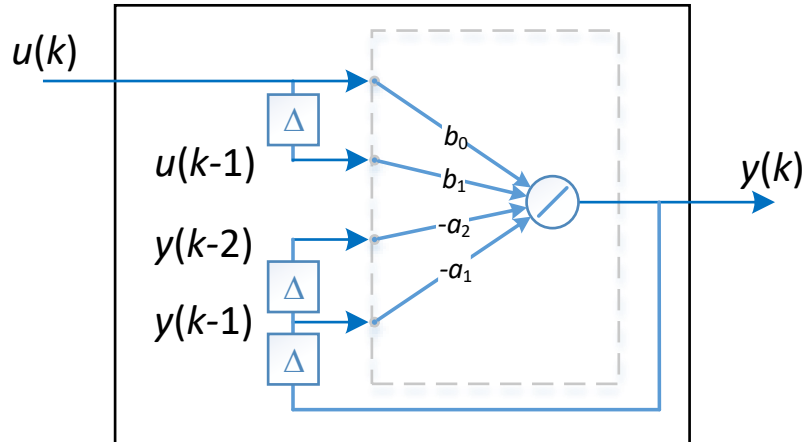
Dinamički model preko FF VNM sa kašnjenjima signala

- Da bi se napravio dinamički model, gde izlaz zavisi od ranijih vrednosti ulaza (i izlaza) može se upotrebiti *feed-forward* VNM, ali pored tekućih ulaza u sistem na ulaze VNM treba dodati ranije vrednosti ulaza i izlaza (proširiti ulaze u VNM).

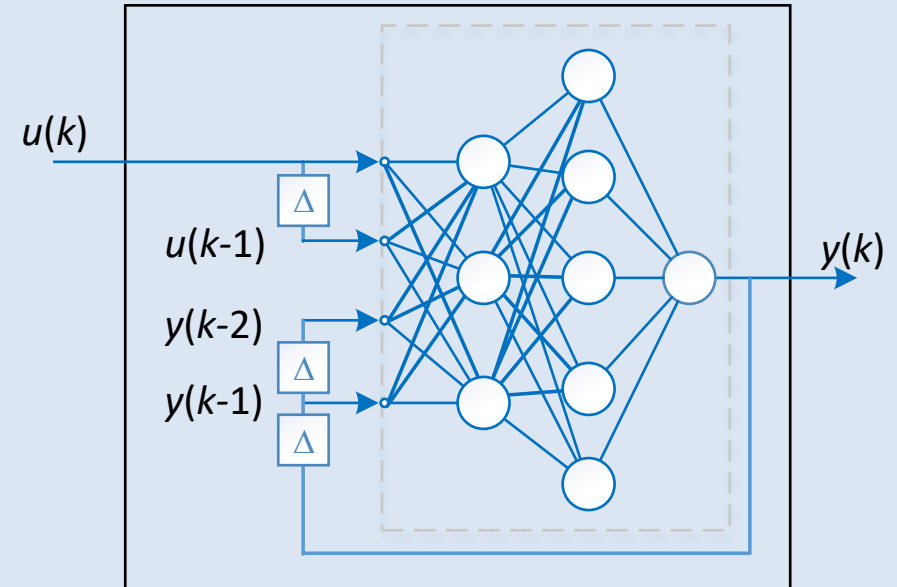


Dinamički model - primer

- Posmatra se model opisan **linearnom** diferencnom jednačinom:
$$y(k) = -a_1y(k-1) - a_2y(k-2) + b_0u(k) + b_1u(k-1)$$
- Model se može realizovati upotrebom jednog neurona sa linearnom aktivacionom funkcijom.



Ideja: Ako se umesto jednog neurona upotrebi složena *feed-forward* mreža sa nelinearnim aktivacionim funkcijama, onda se može modelovati znatno složenije ponašanje, uključujući nelinearne diferencne jednačine.

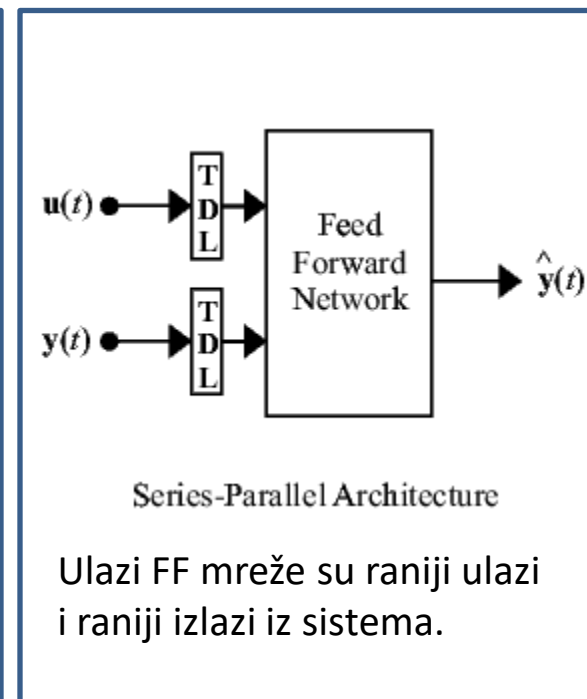
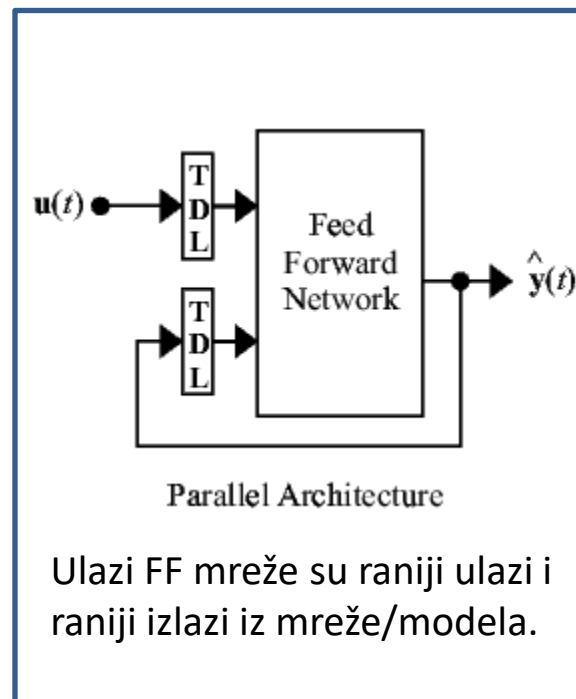
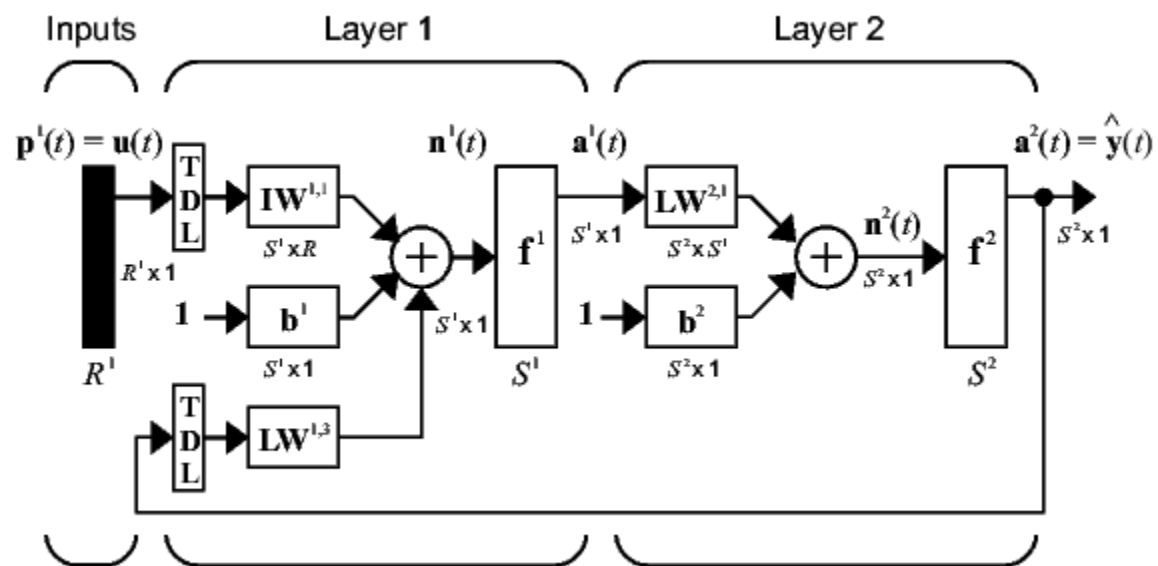


NARX tip mreže

(preuzeto iz Matlab dokumentacije)

NARX = nelinearni ARX tip modela

$$y(t) = f(y(t-1), y(t-2), \dots, y(t-n), u(t-1), u(t-2), \dots, u(t-m))$$



TDL - predstavlja zakašnjene vrednosti signala (*time-delay*)